MGMTMSA 440 Building LLM Applications, Capstone

Fueli: an AI meal and workout planner that cooks from your fridge

Group 1: Rou (Sara) Sun, Hsinyen (Mia) Wu, Yiu-Hon (Jeremy) Lee,
Thanapol (Neil) Doungsaeng, Emmet Andrews

Live app: https://fueli.vercel.app | GitHub: https://github.com/hymiawu/fueli

1. Problem and target user

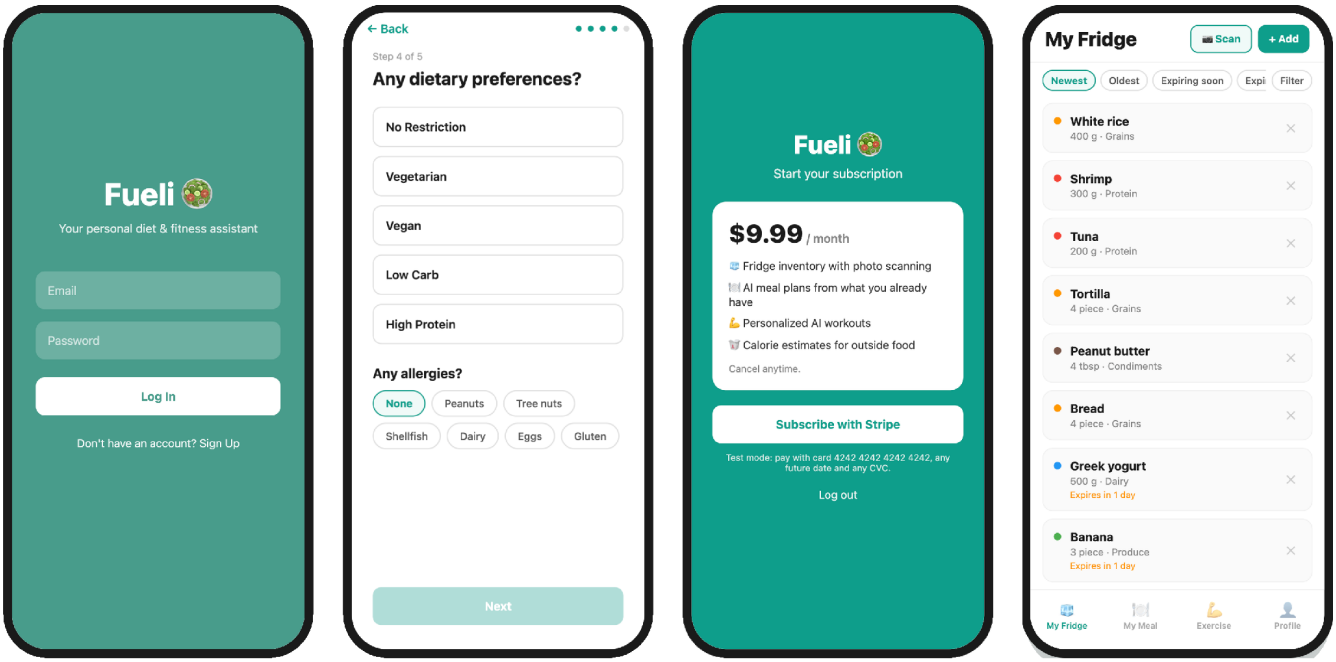
Fueli is for people who cook at home and want to eat and train better. Some already have a goal such as losing fat or building muscle; others are beginners who do not yet cook or exercise regularly but want to start. Many also follow a diet or have a food allergy.

Today, meal planning and fitness tracking live in separate apps. Recipe apps send people shopping while the food in the fridge expires, and a chatbot gives calorie numbers that look precise and are often wrong. It's 7 pm, and your yogurt and bananas expire tomorrow; finding a meal that uses them and fits a muscle-building goal takes longer than ordering takeout.

Fueli plans a day of meals from the user's fridge, checks every calorie number against a nutrition database, and plans the week's training for the same goal. AI reads food labels and generates meal suggestions when no stored recipe fits, while anything requiring exact results is handled by deterministic code. It costs $9.99 a month.

2. What Fueli does

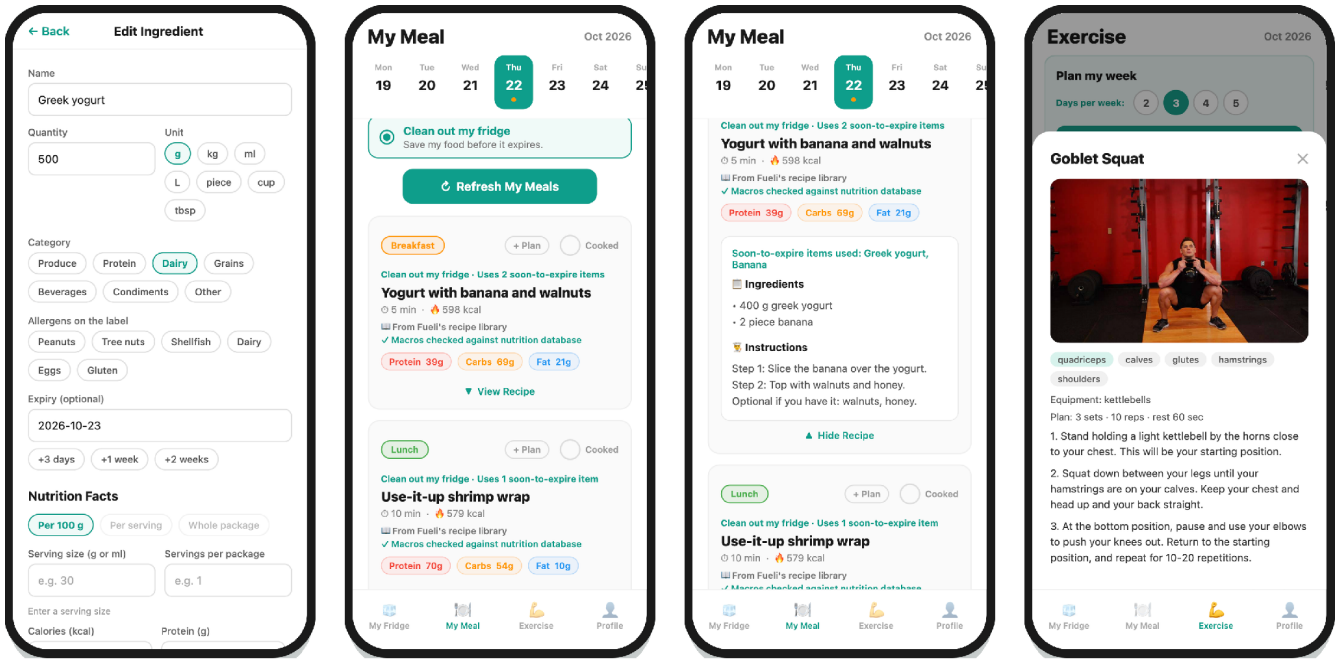
The core journey (login, an AI feature, checkout) works end to end on the live site:



(a) Log in (b) Diet and allergies (c) $9.99 paywall (d) My Fridge

Figure 1. *Sign-up, subscription and the fridge (screens from the live app, October 22, 2026)*

1. **Sign up and subscribe.** A five-step profile (body stats, goal, diet and six allergens), then Stripe Checkout (in test mode for assignment). A signed webhook unlocks the AI features.
2. **Stock My Fridge.** Add items by hand or photograph a label; the vision model reads name, quantity, nutrition and allergens, and the user confirms.
3. **Plan meals.** Pick one of three modes: Feeling hungry (20 minutes or less), Feeling healthy (within 20% of the calorie target) or Clean out my fridge (use what expires within three days).
4. **Cook.** Ticking "Cooked" subtracts the ingredients from the fridge.
5. **Train.** A weekly plan of 2 to 5 training days from a library of 57 exercises with photos and form notes.



(e) Fridge item (f) Fridge plan (g) Recipe (h) Exercise

Figure 2. *The AI features: a fridge item, a Clean out my fridge plan, its recipe, and an exercise*

Figure 2 shows the AI features in use. A scanned item (e) keeps its allergens and its nutrition per 100 g, per serving and per package, and the list warns when something expires soon. A Clean out my fridge plan (f) builds the day around the items that expire first, says which fridge items each meal uses, and marks the meals whose macros were checked against the database. The recipe view (g) gives the steps and a per-serving breakdown, and anything missing goes on a shopping list. On the training side (h), each exercise has photos and step-by-step form notes, and the user logs sets, reps and weight.

In daily use the loop is short: open the fridge, pick a mode, cook, tick Cooked, and the fridge updates itself. One profile drives both plans, so a user with a muscle-building goal gets meals that lean toward protein and a training week built for the same goal. The Membership screen shows the renewal date and how much of today's usage caps is left.

3. System design

3.1 Architecture

The web app is an Expo (React Native) export on Vercel. It signs users in with Supabase Auth and reads the fridge under row-level security (RLS), so users see only their own rows. AI requests go to an Express API on Fly.io, because model calls can take about 10 seconds and because Fly keeps every secret on the server. The coordinator checks the login token, the subscription, the daily caps and the request schema, then calls one of three agents. All model calls use Claude Haiku 5.5.

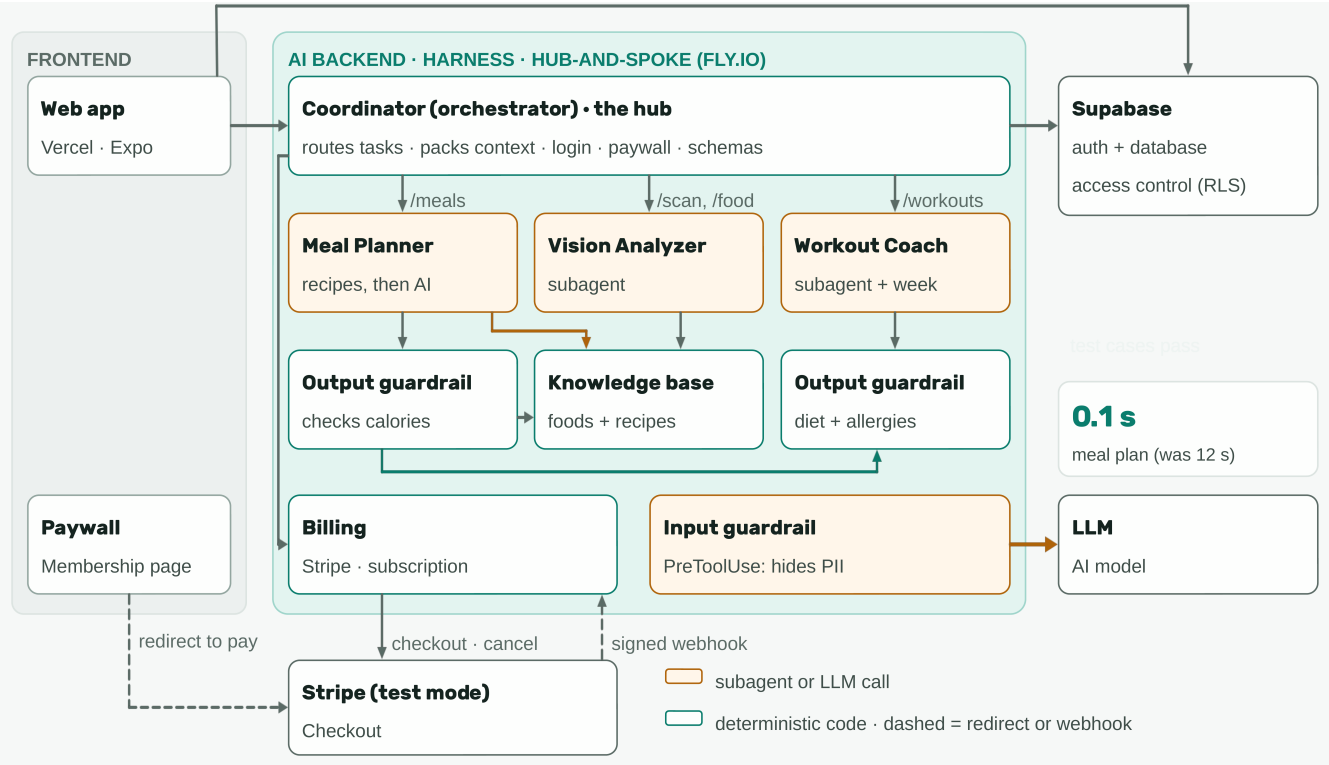


Figure 3. *Fueli system design: a coordinator (hub) on Fly.io with three AI spokes and deterministic checks*

3.2 Where we use AI and where we use code

Task	Approach	Why
Read a food label	Vision model	Open-ended recognition
Match the fridge to a recipe	Code over a 180-recipe library	Fast, free and repeatable
Write a meal no recipe fits	LLM agent with a nutrition tool	Creative composition
Calories and macros	Lookup in a 62-food USDA-based database	Facts should not be generated
Diet, allergy and mode rules	Code, with one repair round	Must never fail silently
Daily workout	Code over a 40-workout library	Real exercises with photos
Weekly training plan	AI, then checked by rules	Two rest days a week

3.3 How a meal plan is made

The planner matches the fridge against the recipe library first; a plan built only from library recipes returns in about 0.1 second with no model call. The AI writes a meal only when no library recipe fits the ingredients. A Nutrition Verifier then recomputes calories and macros from the database whenever at least 70% of a meal's ingredients resolve, and overrides the model when its figure is off by more than 25%. A Safety Guard enforces the diet, the six allergens and the chosen mode's rules. Problems from both checks go into one shared repair round, so a plan costs at most two model calls; a meal that still breaks a rule is held back and shown with the reason.

Errors end in a plain message: a reply that breaks the JSON schema is retried once, a refusal gets a short explanation, an unpaid account goes to the payment screen (HTTP 402), a user over a cap is told to try later (HTTP 429), and our own failures do not count against the cap.

3.4 Privacy and safety

An input guardrail runs before any model call. Name, email, height and weight are never sent to the model, emails and phone numbers typed into free text are redacted, and photo metadata is stripped. Gender only changes the calorie target in code. Fridge item names are wrapped as data, so a prompt injection hidden in an item name is flagged. The model's tools are read-only and scoped by RLS, so a successful injection could at worst change suggestion text, never data. The user confirms every scan, and only a Cooked tick changes the fridge. Card details never touch our servers.

3.5 Business model and cost

Fueli costs $9.99 a month. Each account has daily caps of 20 meal plans, 10 workout requests and 50 scans or food estimates, reset at midnight UTC, plus a rate limit of 30 AI calls per 10 minutes. At the caps a user would cost us about $4.50 a month in model calls. A typical user costs about 30 cents.

4. Evaluation

We wrote the 25-case eval suite before tuning the prompts. It includes adversarial cases (a vegetarian with chicken in the fridge, a peanut allergy, prompt injections); a rubric checks schema, diet, allergy, macro and mode rules. The last full run on Claude Haiku 5.5 passed 25 of 25:

Suite	Cases	Passed	Median latency
Meal plans	12	12	11.5 s
Outside food	6	6	1.0 s
Workouts	5	5	12.9 s
Weekly plan	2	2	3.1 s

With the recipe library the 12 meal cases pass in 0.1 to 0.9 seconds, and all 83 unit tests pass on the final commit (e8dd82c). Two failures gave us the most insight. A breakfast claimed 523 kcal while the database said 688, so now the database always wins when it can evaluate the meal. An injection case passed while the model was copying our example JSON, so we now read outputs as well as scores.

5. Lessons learned and future improvements

5.1 Lessons learned

- **Let the model propose, not decide.** The model drafts and code checks. Calories come from a database, diet and allergy rules are code, and the model's own numbers are kept only for comparison. Once we stopped asking the model to be right about facts, its mistakes stopped reaching users.
- **The fastest model call is the one you skip.** Our biggest speed and cost gain came from not calling the model: matching a recipe library in code took meal plans from about 10 seconds to about 0.1 second, at no token cost. Model choice came second (Haiku over a larger model, 47 seconds to about 10).
- **Read outputs, not checkmarks.** The injection case passed while the model was copying our example JSON. A pass rate tells you what the rubric checked, not what the user sees, so we now read a sample of outputs after every eval run.
- **Write decisions down before building.** Five people's specs went into one handoff document that each person edited in turn, and our Claude Code session applied it in one pull request (#19). Fifty numbered decisions meant the code agent never had to guess what we wanted.
- **Check free-tier limits early.** Supabase's free tier sends two sign-up emails an hour, which we found in a team meeting days before the demo; the demo became invite-only.

5.2 Future improvements

- **Measure scan accuracy.** Label eight to ten photos so label-reading accuracy can be measured.
- **History in the database.** Move meal and workout history from the device to Supabase so the planner can learn from it.
- **Production readiness.** Use the USDA FoodData Central API, custom sign-up email and caps stored in Redis.

6. Team and use of AI tools

- **Hsinyen (Mia) Wu:** Built the app, its API and agents, Supabase and Stripe, and both deployments; merged all pull requests from the rest of the team.
- **Yiu-Hon (Jeremy) Lee:** Evaluated and updated the repo, wrote the decisions and handoff document, built the final slides and recorded video.
- **Thanapol (Neil) Doungsaeng:** Evaluated and updated the repo, built the 57-exercise library, the weekly plan rules and the exercise evals.
- **Rou (Sara) Sun:** Evaluated and updated the repo, wrote the three meal-mode rules and the mode-selection design.
- **Emmet Andrews:** Evaluated and updated the repo, worked on the recipe library, signup flow, and the per-serving nutrition view.

We built Fueli with Claude Code: it moved the prototype's browser-side API key to the server, wrote the eval suite and applied our written specs. The repo has two shared skills (run-evals, deploy-fueli) and two read-only review agents (nutrition-auditor, safety-reviewer). The product decisions were ours, and every change went through a pull request.